ADVANCED PROGRAMMING (MSC) / 11 NOVEMBER 2026

Personal Finance Tracker

Step-by-step task plan v6.1

51 numbered tasks with named owners, React deliverables and paste-ready AI-agent prompts.

Erti Hoxha | Documentation, filters, exports and project management

Isa Bilalli | Repository setup, domain, validation and budgets

Florent Latifi | Accounts, storage, reports and integration

Project dates

11 Nov kickoff at 13:00–14:00 CET. 20 Nov core milestone. 27 Nov code freeze. 30 Nov rehearsal. 1 Dec final check. 2 Dec planned submission. 6 Dec official deadline in the source guide. 8 Dec presentation and defense.

Technology stack

Frontend: Vite + React and Recharts. Backend: Python 3.11+ and FastAPI. Database: a local SQLite file accessed through Python sqlite3 and the Repository interface. Use pytest, ruff and mypy for Python checks and GitHub Actions for CI on main.

The frontend has four screens: Dashboard, Transactions, Budgets and Accounts. Task #05 builds the first Dashboard and configures SQLite. Keep course-required CLI behavior unless the professor approves substitution.

Access model

The course demo runs locally for one user with synthetic data, so it opens directly to the Dashboard. React sends changes to FastAPI, which saves them to the SQLite file. The file retains records across refreshes and restarts. Login can be added for hosted access. Add signup later if users need to register themselves.

What changed from v5.1

Named owners, a React outcome for every task, AI-agent prompts, updated kickoff and main-only Git workflow. Original dates and estimates are planning baselines unless revised here. Re-estimate the added frontend effort at the kickoff.

Workflow and Definition of Done

GitHub uses only main

Use main as the sole GitHub branch. Coordinate one writer per shared area. Pull or fetch the latest main before work and again before pushing. Review a diff with the assigned reviewer before pushing small commits. Use clear commit messages. Run local checks before pushing and check CI on main. Never force-push.

Erti reviews Isa. Isa reviews Florent. Florent reviews Erti. Record reviewer approval in the task card or meeting notes. Tag the verified main commit v1.0-freeze at the freeze.

React delivery checks

The relevant screen or evidence satisfies the task's React deliverable. For UI changes, verify keyboard access, labels, visible focus, narrow-width layout and applicable loading, empty, failure and success states. Use server-calculated totals, stable IDs and decimal amount strings. Test with synthetic data.

Engineering and AI use

Meet the task-specific acceptance checks and confirm the prerequisites on main. Keep public Python code typed and documented. Run pytest, ruff and mypy plus React lint/build and the relevant browser checks. A teammate reviews changes and can explain the implementation.

Paste one prompt at a time into the AI agent and give it the corresponding task details. Review the diff and test results yourself. Record relevant AI assistance and sources, and use synthetic data and placeholders.

Working agreements

Use the Trello card number when coordinating. Raise blockers in the group chat the same day. Defer Could features first, then agreed Should features. Confirm any change to a Must requirement with the professor.

Frontend work packages

Setup and contracts: #05, #10, #17, #21.

Accounts and balances: #07, #11, #18.

Transactions, filters, transfers and recurrence: #06, #08, #12, #14, #15, #23, #25.

Dashboard and budgets: #19, #26, #27, #31.

Exports and integration: #16, #24, #32, #36, #37.

Task index

Task	Title	Owner	Due
#01	Confirm the group in Topics.xlsx	Erti Hoxha	9 Nov
#02	Share this guide and set up the task board	Erti Hoxha	9 Nov
#03	Read the guide and confirm assigned responsibilities	All three	10 Nov
#04	Kickoff meeting	All three	11 Nov
#05	Project setup: repository, tools and skeleton	Isa Bilalli	12 Nov
#06	Money value object	Isa Bilalli	14 Nov
#07	Account class	Florent Latifi	14 Nov
#08	Composable filters	Erti Hoxha	15 Nov
#09	Documentation: create the shared document and write Section 1	Erti Hoxha	17 Nov
#10	Enums and exception hierarchy	Isa Bilalli	14 Nov
#11	Ledger: accounts, events and Observer	Florent Latifi	15 Nov
#12	Validators: descriptors and decorators	Isa Bilalli	15 Nov
#13	Check-in #1	All three	13 Nov
#14	Transaction hierarchy: Income, Expense, Transfer	Isa Bilalli	17 Nov
#15	Sorting and text search	Erti Hoxha	17 Nov
#16	Exporter base class and CsvExporter	Erti Hoxha	18 Nov
#17	TransactionFactory	Isa Bilalli	18 Nov
#18	Repository and atomic save	Florent Latifi	18 Nov
#19	Report base class and MonthlySummary	Florent Latifi	20 Nov
#20	Documentation: Section 2, first draft	Erti Hoxha	21 Nov
#21	First CLI commands	Florent Latifi	20 Nov
#22	Milestone W1 review: the core works	All three	20 Nov
#23	Transfers between accounts	Isa Bilalli	21 Nov
#24	JSON and Markdown exporters + CSV injection guard	Erti Hoxha	22 Nov
#25	Recurring transactions	Isa Bilalli	22 Nov
#26	CategoryReport	Florent Latifi	22 Nov

Task index continued

Task	Title	Owner	Due
#27	Budgets and BudgetMonitor (Observer)	Isa Bilalli	24 Nov
#28	Optional (Could) features, only if decided at #22	Erti Hoxha (scope coordinator)	27 Nov
#29	Documentation input for Section 3: the domain	Isa Bilalli	24 Nov
#30	Code walk-through	All three	24 Nov
#31	TrendReport and BudgetReport	Isa Bilalli	25 Nov
#32	Export commands in the CLI	Erti Hoxha	25 Nov
#33	Documentation: finish Section 2	Erti Hoxha	25 Nov
#34	Documentation input for Section 3: storage, reports, CLI	Florent Latifi	25 Nov
#35	Check-in #2	All three	25 Nov
#36	Full CLI menu and integration	Florent Latifi	27 Nov
#37	Finish tests: 60+ tests and 80% coverage	All three	27 Nov
#38	Slide content: slides 4 and 5	Isa Bilalli	27 Nov
#39	Slide content: slides 6, 7 and 8	Florent Latifi	27 Nov
#40	CODE FREEZE	All three	27 Nov
#41	Final figures and UML diagrams	Erti Hoxha	28 Nov
#42	Documentation: Section 3 Implementation	Erti Hoxha	29 Nov
#43	Documentation: Section 4 Conclusion and Section 5 References	Erti Hoxha	29 Nov
#44	Demo video and backup screenshots	Erti Hoxha	28 Nov
#45	Slides: build the full deck	Erti Hoxha	29 Nov
#46	Cross proofreading	All three	29 Nov
#47	Documentation: formatting and final proofreading	Erti Hoxha	30 Nov
#48	Rehearsal #1 (Milestone D: docs and slides ready)	All three	30 Nov
#49	FINAL CHECK: the project is finished	All three	1 Dec
#50	Submit the project	Erti Hoxha	2 Dec
#51	After submission: defense preparation	All three	8 Dec

Task timeline

Baseline work windows in November and December 2026. Colors identify the owner. Check task prerequisites before starting integration.

Task	Work item	09 Nov	16 Nov	23 Nov	30 Nov	07 Dec
#01	Confirm the group in Topics.xlsx	9				
#02	Share this guide and set up the task board	9				
#03	Read the guide and confirm assigned responsibilities	9–10				
#04	Kickoff meeting	11				
#05	Project setup: repository, tools and skeleton	11–12				
#06	Money value object	12–14				
#07	Account class	12–14				
#08	Composable filters	12–15				
#09	Documentation: create the shared document and write Section 1	14–15	16–17			
#10	Enums and exception hierarchy	14				
#11	Ledger: accounts, events and Observer	14–15				
#12	Validators: descriptors and decorators	14–15				
#13	Check-in #1	13				
#14	Transaction hierarchy: Income, Expense, Transfer	15	16–17			
#15	Sorting and text search		17			
#16	Exporter base class and CsvExporter		17–18			
#17	TransactionFactory		17–18			
#18	Repository and atomic save		17–18			
#19	Report base class and MonthlySummary		17–20			
#20	Documentation: Section 2, first draft		18–21			
#21	First CLI commands		18–20			
#22	Milestone W1 review: the core works		20			
#23	Transfers between accounts		21			
#24	JSON and Markdown exporters + CSV injection guard		21–22			
#25	Recurring transactions		21–22			
#26	CategoryReport		22			

Task timeline continued

Baseline work windows in November and December 2026. Colors identify the owner. Check task prerequisites before starting integration.

Task	Work item	09 Nov	16 Nov	23 Nov	30 Nov	07 Dec
#27	Budgets and BudgetMonitor (Observer)		22	23–24		
#28	Optional (Could) features, only if decided at #22		22	23–27		
#29	Documentation input for Section 3: the domain			24		
#30	Code walk-through			24		
#31	TrendReport and BudgetReport			24–25		
#32	Export commands in the CLI			24–25		
#33	Documentation: finish Section 2			25		
#34	Documentation input for Section 3: storage, reports, CLI			25		
#35	Check-in #2			25		
#36	Full CLI menu and integration			25–27		
#37	Finish tests: 60+ tests and 80% coverage		21–22	23–27		
#38	Slide content: slides 4 and 5			27		
#39	Slide content: slides 6, 7 and 8			27		
#40	CODE FREEZE			27		
#41	Final figures and UML diagrams			28		
#42	Documentation: Section 3 Implementation			28–29		
#43	Documentation: Section 4 Conclusion and Section 5 References			29		
#44	Demo video and backup screenshots			28		
#45	Slides: build the full deck			25–29		
#46	Cross proofreading			29		
#47	Documentation: formatting and final proofreading				30	
#48	Rehearsal #1 (Milestone D: docs and slides ready)				30	
#49	FINAL CHECK: the project is finished				1	
#50	Submit the project				2	
#51	After submission: defense preparation				3–6	7–8

Daily coordination

Asterisk marks a task due that day. Shared tasks apply to all three. Erti also coordinates #28 if the optional scope is approved.

Day	Erti Hoxha	Isa Bilalli	Florent Latifi	Shared
Mon 09 Nov	#01* #02*	—	—	#03
Tue 10 Nov	—	—	—	#03*
Wed 11 Nov	—	#05	—	#04*
Thu 12 Nov	#08	#05* #06	#07	—
Fri 13 Nov	#08	#06	#07	#13*
Sat 14 Nov	#08 #09	#06* #10* #12	#07* #11	—
Sun 15 Nov	#08* #09	#12* #14	#11*	—
Mon 16 Nov	#09	#14	—	—
Tue 17 Nov	#09* #15* #16	#14* #17	#18 #19	—
Wed 18 Nov	#16* #20	#17*	#18* #19 #21	—
Thu 19 Nov	#20	—	#19 #21	—
Fri 20 Nov	#20	—	#19* #21*	#22*
Sat 21 Nov	#20* #24	#23* #25	—	#37
Sun 22 Nov	#24* #28	#25* #27	#26*	#37
Mon 23 Nov	#28	#27	—	#37

Daily coordination continued

Asterisk marks a task due that day. Shared tasks apply to all three. Erti also coordinates #28 if the optional scope is approved.

Day	Erti Hoxha	Isa Bilalli	Florent Latifi	Shared
Tue 24 Nov	#28 #32	#27* #29* #31	—	#30* #37
Wed 25 Nov	#28 #32* #33* #45	#31*	#34* #36	#35* #37
Thu 26 Nov	#28 #45	—	#36	#37
Fri 27 Nov	#28* #45	#38*	#36* #39*	#37* #40*
Sat 28 Nov	#41* #42 #44* #45	—	—	—
Sun 29 Nov	#42* #43* #45*	—	—	#46*
Mon 30 Nov	#47*	—	—	#48*
Tue 01 Dec	—	—	—	#49*
Wed 02 Dec	#50*	—	—	—
Thu 03 Dec	—	—	—	#51
Fri 04 Dec	—	—	—	#51
Sat 05 Dec	—	—	—	#51
Sun 06 Dec	—	—	—	#51
Mon 07 Dec	—	—	—	#51
Tue 08 Dec	—	—	—	#51*

Erti Hoxha task checklist

Original v5.1 estimate including shared tasks: 55.5 h + 2 h of project management tasks. React work requires a separate estimate.

Task	Work item	Due	Track
#01	Confirm the group in Topics.xlsx	09 Nov	☐
#02	Share this guide and set up the task board	09 Nov	☐
#03	Read the guide and confirm assigned responsibilities	10 Nov	☐ Shared
#04	Kickoff meeting	11 Nov	☐ Shared
#08	Composable filters	15 Nov	☐
#09	Documentation: create the shared document and write Section 1	17 Nov	☐
#13	Check-in #1	13 Nov	☐ Shared
#15	Sorting and text search	17 Nov	☐
#16	Exporter base class and CsvExporter	18 Nov	☐
#20	Documentation: Section 2, first draft	21 Nov	☐
#22	Milestone W1 review: the core works	20 Nov	☐ Shared
#24	JSON and Markdown exporters + CSV injection guard	22 Nov	☐
#28	Optional (Could) features, only if decided at #22	27 Nov	☐
#30	Code walk-through	24 Nov	☐ Shared
#32	Export commands in the CLI	25 Nov	☐
#33	Documentation: finish Section 2	25 Nov	☐
#35	Check-in #2	25 Nov	☐ Shared
#37	Finish tests: 60+ tests and 80% coverage	27 Nov	☐ Shared
#40	CODE FREEZE	27 Nov	☐ Shared
#41	Final figures and UML diagrams	28 Nov	☐
#42	Documentation: Section 3 Implementation	29 Nov	☐
#43	Documentation: Section 4 Conclusion and Section 5 References	29 Nov	☐
#44	Demo video and backup screenshots	28 Nov	☐
#45	Slides: build the full deck	29 Nov	☐
#46	Cross proofreading	29 Nov	☐ Shared
#47	Documentation: formatting and final proofreading	30 Nov	☐
#48	Rehearsal #1 (Milestone D: docs and slides ready)	30 Nov	☐ Shared
#49	FINAL CHECK: the project is finished	01 Dec	☐ Shared
#50	Submit the project	02 Dec	☐
#51	After submission: defense preparation	08 Dec	☐ Shared

Project management

Lead the kickoff, check-ins, reviews and walk-through. Keep Trello current, coordinate blockers and freeze tagging, and act as the professor contact.

Isa Bilalli task checklist

Original v5.1 estimate including shared tasks: 53.5 h. React work requires a separate estimate.

Task	Work item	Due	Track
#03	Read the guide and confirm assigned responsibilities	10 Nov	☐ Shared
#04	Kickoff meeting	11 Nov	☐ Shared
#05	Project setup: repository, tools and skeleton	12 Nov	☐
#06	Money value object	14 Nov	☐
#10	Enums and exception hierarchy	14 Nov	☐
#12	Validators: descriptors and decorators	15 Nov	☐
#13	Check-in #1	13 Nov	☐ Shared
#14	Transaction hierarchy: Income, Expense, Transfer	17 Nov	☐
#17	TransactionFactory	18 Nov	☐
#22	Milestone W1 review: the core works	20 Nov	☐ Shared
#23	Transfers between accounts	21 Nov	☐
#25	Recurring transactions	22 Nov	☐
#27	Budgets and BudgetMonitor (Observer)	24 Nov	☐
#29	Documentation input for Section 3: the domain	24 Nov	☐
#30	Code walk-through	24 Nov	☐ Shared
#31	TrendReport and BudgetReport	25 Nov	☐
#35	Check-in #2	25 Nov	☐ Shared
#37	Finish tests: 60+ tests and 80% coverage	27 Nov	☐ Shared
#38	Slide content: slides 4 and 5	27 Nov	☐
#40	CODE FREEZE	27 Nov	☐ Shared
#46	Cross proofreading	29 Nov	☐ Shared
#48	Rehearsal #1 (Milestone D: docs and slides ready)	30 Nov	☐ Shared
#49	FINAL CHECK: the project is finished	01 Dec	☐ Shared
#51	After submission: defense preparation	08 Dec	☐ Shared

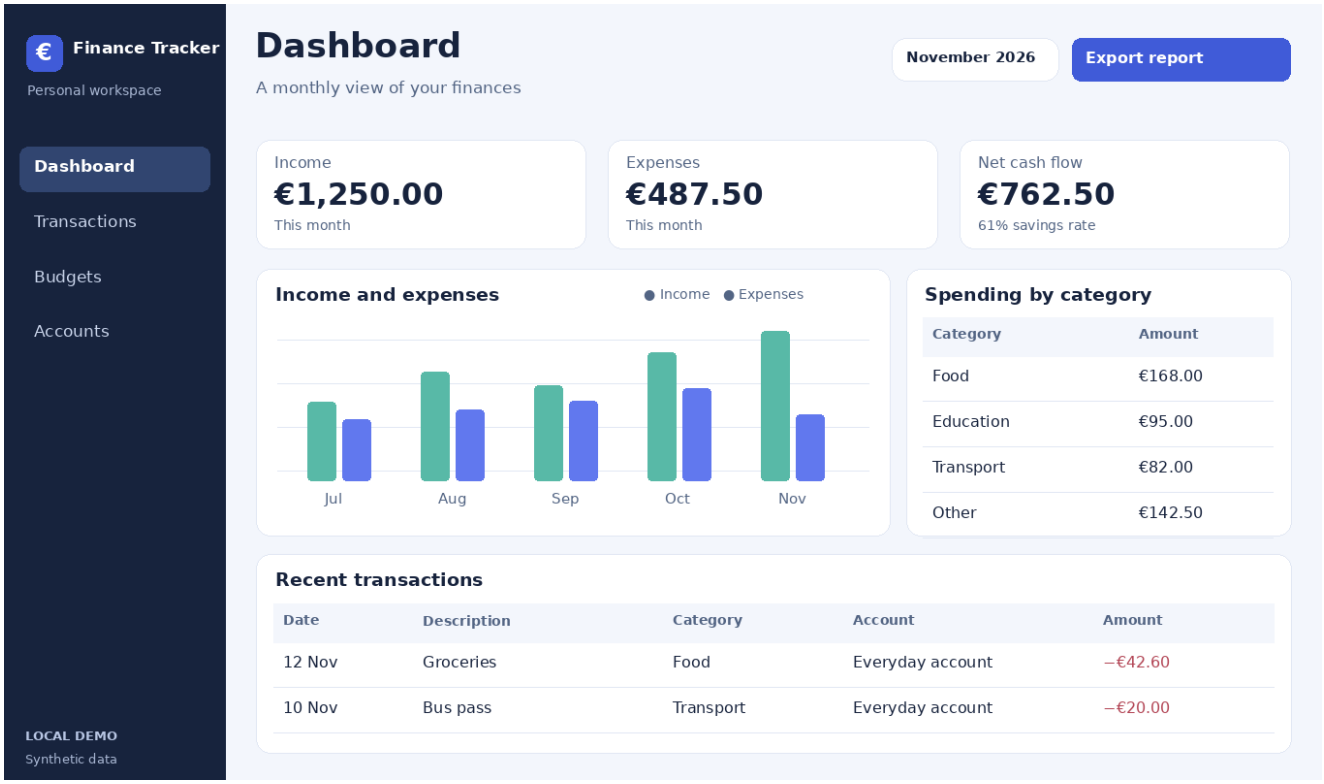
Florent Latifi task checklist

Original v5.1 estimate including shared tasks: 48.5 h. React work requires a separate estimate.

Task	Work item	Due	Track
#03	Read the guide and confirm assigned responsibilities	10 Nov	☐ Shared
#04	Kickoff meeting	11 Nov	☐ Shared
#07	Account class	14 Nov	☐
#11	Ledger: accounts, events and Observer	15 Nov	☐
#13	Check-in #1	13 Nov	☐ Shared
#18	Repository and atomic save	18 Nov	☐
#19	Report base class and MonthlySummary	20 Nov	☐
#21	First CLI commands	20 Nov	☐
#22	Milestone W1 review: the core works	20 Nov	☐ Shared
#26	CategoryReport	22 Nov	☐
#30	Code walk-through	24 Nov	☐ Shared
#34	Documentation input for Section 3: storage, reports, CLI	25 Nov	☐
#35	Check-in #2	25 Nov	☐ Shared
#36	Full CLI menu and integration	27 Nov	☐
#37	Finish tests: 60+ tests and 80% coverage	27 Nov	☐ Shared
#39	Slide content: slides 6, 7 and 8	27 Nov	☐
#40	CODE FREEZE	27 Nov	☐ Shared
#46	Cross proofreading	29 Nov	☐ Shared
#48	Rehearsal #1 (Milestone D: docs and slides ready)	30 Nov	☐ Shared
#49	FINAL CHECK: the project is finished	01 Dec	☐ Shared
#51	After submission: defense preparation	08 Dec	☐ Shared

Dashboard GUI mockup

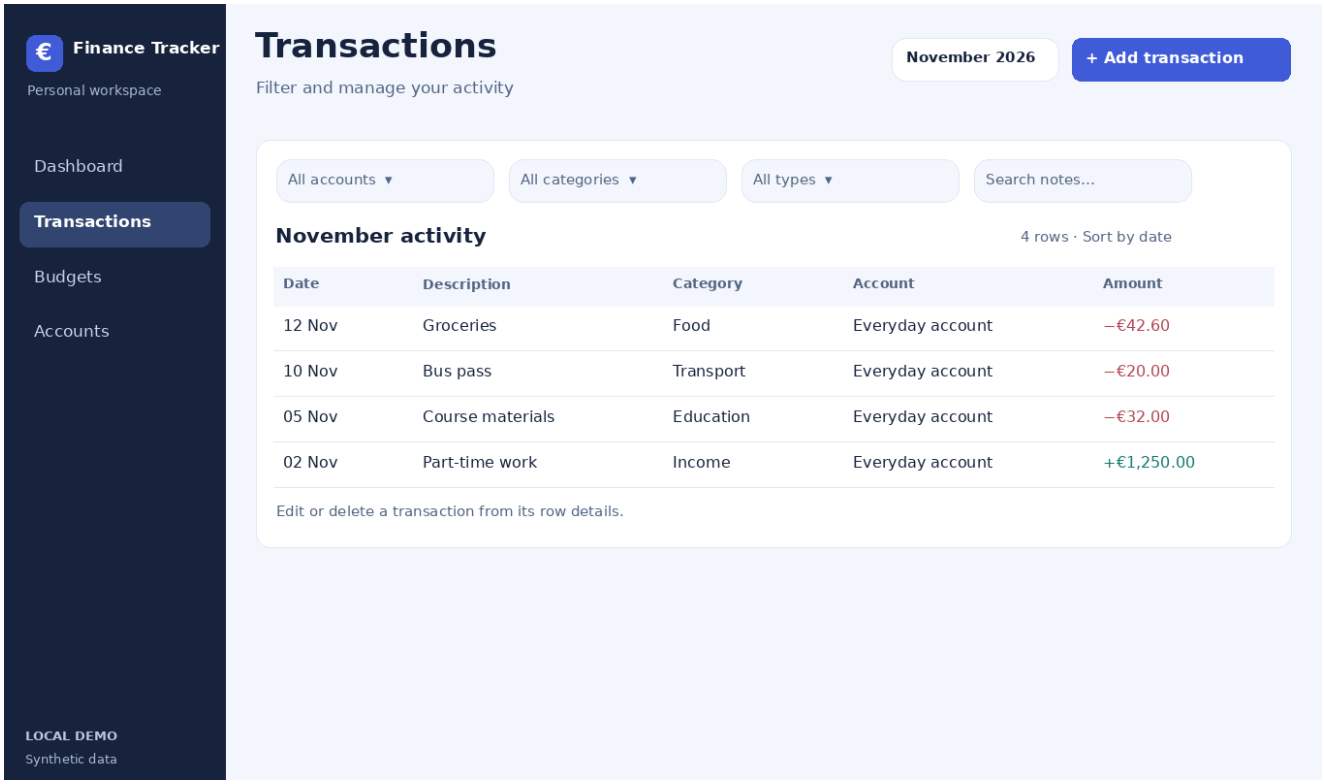
Design reference with synthetic data. The team can adapt the layout to the agreed API.



Monthly summary, spending breakdown and recent transactions.

Transactions GUI mockup

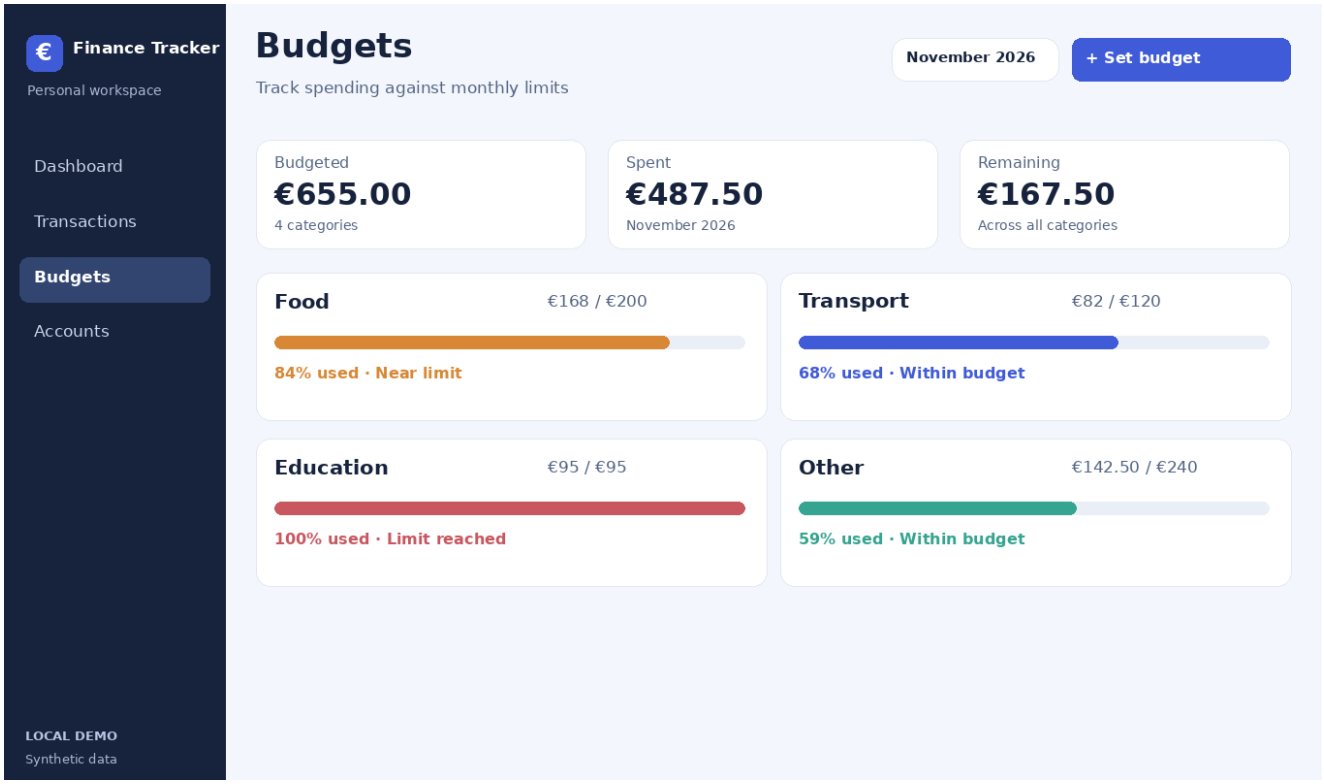
Design reference with synthetic data. The team can adapt the layout to the agreed API.



Search, filtering and transaction management.

Budgets GUI mockup

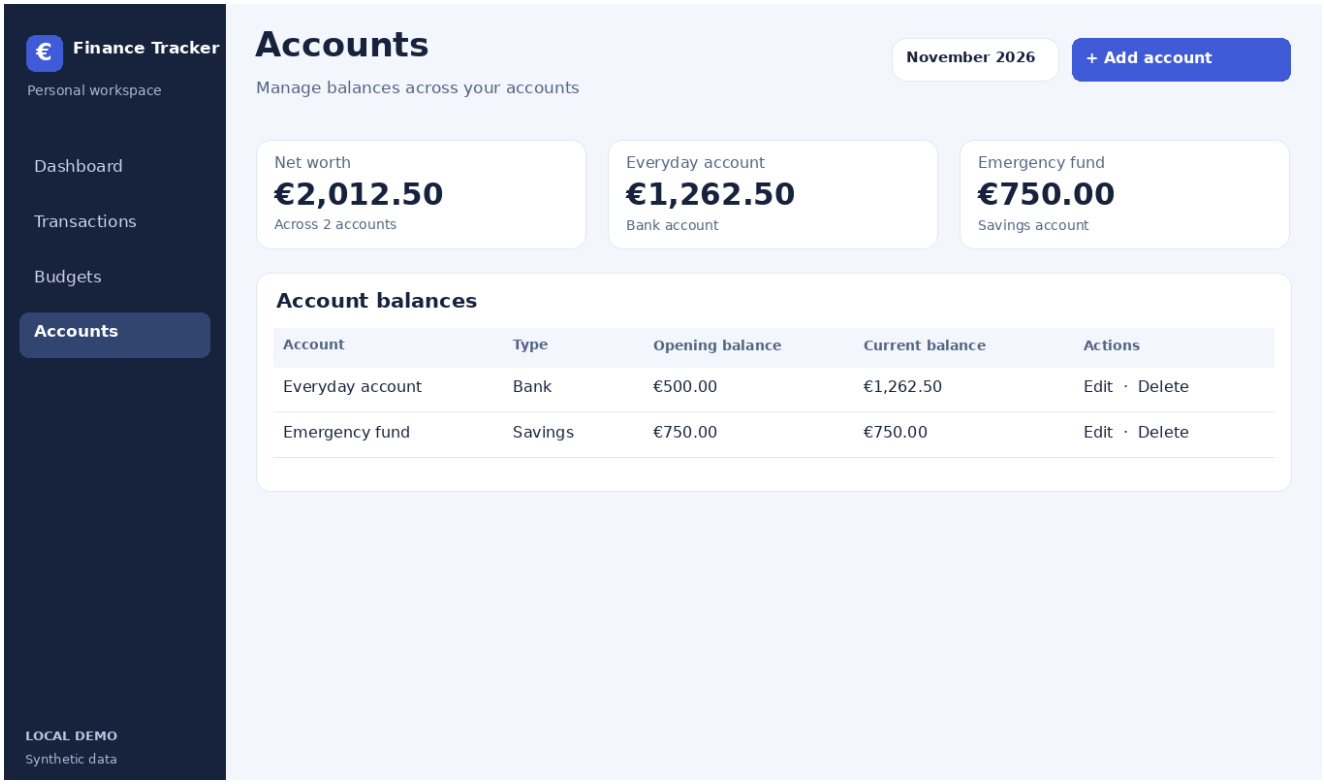
Design reference with synthetic data. The team can adapt the layout to the agreed API.



Monthly progress with readable threshold states.

Accounts GUI mockup

Design reference with synthetic data. The team can adapt the layout to the agreed API.



Net worth, opening balances and account management.

#01 Confirm the group in Topics.xlsx

Erti Hoxha

Mon 9 Nov Needs — 0.5 h | v5.1 source p. 8

The professor registers topics first-in, first-out, and the deadline is Friday 13 November. Without a confirmed row we have no project.

Task steps

1. Open the shared Topics.xlsx and check that our group and the topic “Personal Finance Tracker” are there.
2. Write the names of all three members in the row.
3. Never delete, move or overwrite the rows of other groups.
4. Take a screenshot of our row and post it in the group chat.

Done when

• Our row is in Topics.xlsx with all names

• Screenshot is in the group chat

Deliverables

Screenshot of Topics.xlsx

React deliverable

Confirm the project name used in the React page title, sidebar and demo documentation matches the registered topic.

AI-agent prompt

Work on #01 (Confirm the group in Topics.xlsx) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Confirm the project name used in the React page title, sidebar and demo documentation matches the registered topic. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#02 Share this guide and set up the task board

Erti Hoxha

Mon 9 Nov Needs #1 1 h | v5.1 source p. 8

Everyone should come to the kickoff having read the plan, and every task should already be on the board.

Task steps

1. Share this revised guide and the existing Trello board with the team.
2. Keep the 51 numbered cards in To do, In progress, In review or Done.
3. Check each card has its named owner, React deliverable and acceptance criteria.
4. Keep the existing checklist and record the kickoff time in the meeting notes.

Done when

- Revised plan is shared.
- All 51 cards have named ownership and frontend requirements.

Deliverables

Shared guide and Trello board link.

React deliverable

Add a React deliverable and acceptance check to every numbered Trello card. Record the four demo routes in the shared guide.

AI-agent prompt

Work on #02 (Share this guide and set up the task board) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Add a React deliverable and acceptance check to every numbered Trello card. Record the four demo routes in the shared guide. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#03 Read the guide and confirm assigned responsibilities

Erti Hoxha, Isa Bilalli, Florent Latifi

Mon 9 Nov – Tue 10 Nov Needs #2 1 h | v5.1 source p. 8

Everyone should understand their assigned work and be ready to run both parts.

Task steps

1. Read the guide and confirm the work assigned to Erti Hoxha, Isa Bilalli and Florent Latifi.
2. Install Python 3.11+, the agreed Node version, Git and an editor.
3. Check local setup and share any blockers.

Done when

• Each person confirms their responsibilities.

• Python, Node and Git work locally.

Deliverables

—

React deliverable

Install the agreed Node version and run the Vite starter locally. Each person checks their assigned screen or integration responsibility.

AI-agent prompt

Work on #03 (Read the guide and confirm assigned responsibilities) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Install the agreed Node version and run the Vite starter locally. Each person checks their assigned screen or integration responsibility. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#04 Kickoff meeting

Erti Hoxha, Isa Bilalli, Florent Latifi

Wed 11 Nov, 13:00–14:00 CET | Needs #3 | 1 h | v5.1 source p. 9

One meeting where we fix the roles, the interfaces and the rules. After it, all three members can work in parallel.

Task steps

1. 13:00–13:12: Review assessment, requirements and scope.
2. 13:12–13:20: Confirm named owners and review pairs.
3. 13:20–13:35: Agree the domain, React screens, API and storage boundaries.
4. 13:35–13:45: Review milestones and dependencies.
5. 13:45–13:53: Confirm main-only workflow and Definition of Done.
6. 13:53–14:00: Record open decisions, next actions and owners.

Done when

- Each person knows their role and their task numbers
- Interface names agreed and written down
- All meeting dates are in everyone’s calendar

Deliverables

Meeting notes in the chat

React deliverable

Confirm Dashboard, Transactions, Budgets and Accounts. Agree API schemas, decimal strings, synthetic data, demo boundaries and who reviews each route.

AI-agent prompt

Work on #04 (Kickoff meeting) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Confirm Dashboard, Transactions, Budgets and Accounts. Agree API schemas, decimal strings, synthetic data, demo boundaries and who reviews each route. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#05 Project setup: repository, tools and skeleton

Isa Bilalli

Wed 11 Nov – Thu 12 Nov Needs #4 3 h original estimate (re-estimate expanded scope) v4: T-01 · NFR-04, NFR-05 | v5.1 source p. 9

This is the first coding task and it blocks everyone else. Do it the same evening as the kickoff if possible.

Task steps

1. Create a private GitHub repository with main as the sole branch. Add the three collaborators.

2. Set up Python 3.11+, FastAPI bound to localhost, Vite + React, Recharts, a shared layout, route stubs and a frontend lockfile.

3. Configure a SQLite file using Python sqlite3. Use data/tracker.sqlite3 by default, allow a path override, keep the file out of Git and add a repeatable initialization command with optional synthetic seed data.

4. Build the Dashboard described below. Use fixture data until #19 wires the live summary. Verify a synthetic SQLite record survives backend restart. Complete domain persistence in #18.

5. Document setup, the database path and demo reset in the README. Run Python checks and React lint/build in CI on main. Review changes before pushing.

Done when

- Python installs and React builds on a fresh clone.
- CI runs on main and is green.
- Every collaborator can run both parts.
- main is the only GitHub branch.
- The Dashboard goes beyond the mockup and works at desktop and narrow widths.
- SQLite initializes on a fresh clone and a saved record survives a backend restart.

Deliverables

Working Dashboard, FastAPI skeleton, SQLite initialization and persistence test, README and CI.

Be ready to explain: Why the local demo skips authentication and how SQLite persists records.

React deliverable

Build the first working Dashboard in Vite + React. Go beyond the sample screenshot with a month picker, summary cards, Recharts income and spending trends, a category breakdown and recent transactions. Make the layout responsive and demonstrate populated, empty, loading and error states using synthetic fixtures. Connect live summary data in #19.

Access and persistence

The demo is local and single-user, so it opens directly to the Dashboard. React sends changes to FastAPI, which commits them to a local SQLite file. The file keeps the records after refresh or restart. Login can be added for hosted access, and signup can be added if users need to register themselves.

AI-agent prompt

Build task #05 for Isa Bilalli. Set up Vite + React, Recharts, Python/FastAPI and file-backed SQLite using sqlite3. Build the Dashboard with synthetic fixtures and all stated UI states. Configure repeatable database setup and verify a saved record survives a restart. Keep the demo local and single-user. Leave live summary integration to #19. Run Python checks and React lint/build. Use only main and show the diff and test results before pushing.

#06 Money value object

Isa Bilalli

Thu 12 Nov – Sat 14 Nov Needs #5 4 h v4: T-02 · NFR-01 | v5.1 source p. 10

Every amount in the system is a Money object, so this class must be 100% correct before anything else uses it.

Task steps

1. Create Money as @dataclass(frozen=True, slots=True) with amount: Decimal and currency: str = 'EUR'.

2. In __post_init__: reject float with a clear error, convert int/str to Decimal and round to 2 decimals with ROUND_HALF_UP.

3. Add operators: +, −, unary −, multiplication by int/Decimal; comparisons with @total_ordering and __lt__.

4. Add Money.zero(), __radd__ (so sum() works), __str__ → '12.50 EUR', and raise CurrencyMismatchError for different currencies.

5. Write at least 8 tests in tests/unit/test_money.py.

Done when

- Money('0.10') + Money('0.20') == Money('0.30')
- Money('2.675') becomes 2.68; Money(0.1) is rejected
- EUR + USD raises CurrencyMismatchError
- Changing an attribute raises FrozenInstanceError
- sum(items, Money.zero()) works; 8+ tests pass

Deliverables

pft/domain/money.py, tests/unit/test_money.py

Be ready to explain: Value object, dataclasses, operator overloading, why Decimal and not float.

React deliverable

Create a reusable EUR formatter and money input. Keep request amounts as decimal strings, render negative and zero balances, and leave exact calculations to Python.

AI-agent prompt

Work on #06 (Money value object) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Create a reusable EUR formatter and money input. Keep request amounts as decimal strings, render negative and zero balances, and leave exact calculations to Python. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
@total_ordering
@dataclass(frozen=True, slots=True)
class Money:
    amount: Decimal
    currency: str = "EUR"
    def __post_init__(self) -> None:
        if isinstance(self.amount, float):
            raise InvalidAmountError("Use Decimal or str, not float")
        q = Decimal(str(self.amount)).quantize(
            Decimal("0.01"), rounding=ROUND_HALF_UP)
object.__setattr__(self, "amount", q) # frozen: set once
    def __add__(self, other: "Money") -> "Money":
self._check(other) # CurrencyMismatchError
        return Money(self.amount + other.amount, self.currency)
```

#07 Account class

Florent Latifi

Thu 12 Nov – Sat 14 Nov Needs #5 3 h v4: T-03 · FR-01, FR-04, FR-05 | v5.1 source p. 11

An account (cash, bank, card, savings) owns its transactions and knows its balance. It is the heart of encapsulation in our project.

Task steps

1. Attributes: id, name, type (AccountType), opening_balance (Money).

2. Keep transactions in a private list _transactions. Change it only through add(t) and remove(tx_id); remove raises TransactionNotFoundError.

3. Add balance as a @property: opening balance + sum of t.signed_amount().

4. Add dunder methods: __iter__, __len__, __contains__, __repr__, and rename(new_name).

5. Isa Bilalli's Transaction is not ready yet: write a tiny stub class with signed_amount() in the test file and replace it when #14 is merged.

6. Write at least 6 tests.

7. Add the FR-04 edit use case by transaction ID, preserving balances and atomic transfer updates.

Done when

• New account balance = opening balance →

• Opening 50.00 + expense 12.50 balance 37.50

• Removing an unknown ID raises TransactionNotFoundError

• for t in account and len(account) work; the list cannot be changed from outside

• 6+ tests pass

Deliverables

pft/domain/account.py, tests/unit/test_account.py

Be ready to explain: Encapsulation, @property, composition, dunder methods.

React deliverable

Build the Accounts screen with create, rename and delete, showing type, opening balance and current balance. Wire transaction list, edit and delete by stable ID for FR-04, including confirmation and error states.

AI-agent prompt

Work on #07 (Account class) for Florent Latifi. Read this task and its dependencies first. React deliverable: Build the Accounts screen with create, rename and delete, showing type, opening balance and current balance. Wire transaction list, edit and delete by stable ID for FR-04, including confirmation and error states. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#08 Composable filters

Erti Hoxha

Thu 12 Nov – Sun 15 Nov Needs #5 3 h v4: T-04 · FR-06 | v5.1 source p. 11

The CLI and every report select transactions with these filters, so they must be small, combinable and lazy.

Task steps

1. Write filter builders that return functions (predicates): by_category(*cats), by_kind(kind), by_account(id), between(start, end), amount_between(lo, hi).
2. Write all_of(*preds) and any_of(*preds) to combine them.
3. Write apply_filters(items, *preds) that returns a generator, not a list.
4. Make preset filters with functools.partial, e.g. this_month = partial(between, first_day, last_day).
5. Write at least 6 tests, including one that proves the result is a generator.

Done when

- FOOD + amount above 20 returns only matching transactions
- No filters returns everything; date ranges include both end dates
- The result is a generator (tested)
- 6+ tests pass

Deliverables

pft/services/filters.py, tests/unit/test_filters.py

Be ready to explain: Higher-order functions, closures, generators, functools.

React deliverable

Build combinable category, type, account, date, amount and note filters on Transactions. Include reset, active-filter display and an empty-results state.

AI-agent prompt

Work on #08 (Composable filters) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Build combinable category, type, account, date, amount and note filters on Transactions. Include reset, active-filter display and an empty-results state. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
def by_category(*cats: Category) -> Predicate:
    return lambda t: t.category in cats
def all_of(*preds: Predicate) -> Predicate:
    return lambda t: all(p(t) for p in preds)
def apply_filters(items, *preds) -> Iterator[Transaction]:
    combined = all_of(*preds)
return (t for t in items if combined(t)) # lazy generator
```

#09 Documentation: create the shared document and write Section 1

Erti Hoxha

Sat 14 Nov – Tue 17 Nov Needs #4 3 h v4: D-01 | v5.1 source p. 12

Documentation starts in week 1, together with the code. Section 1 does not depend on code, so it can be written now.

Task steps

1. Create the shared document from the professor’s template (Word or Google Docs). Set Times New Roman 10 pt for all styles: body, headings, tables, captions.

2. Create exactly these five headings: 1. Introduction · 2. Description of the Solution / Approach / Architecture · 3. Implementation · 4. Conclusion · 5. References.

3. Write Section 1 (about 1 page) in four short parts: the problem (people do not know where their money goes); the goal of the project; the scope (what is in and what is out); why Python and OOP.

4. End Section 1 with one paragraph that describes the structure of the document (“Section 2 describes …”).

5. Cite 1–2 sources (e.g. the Python documentation, a book on design patterns) in IEEE style [1].

6. Share the link with everyone in edit or comment mode.

Done when

• Template, font and the 5 headings are set

• Section 1 is about 1 page, simple formal English

• At least 1–2 sources cited as [1], [2]

• Same names as the code (Ledger, Account, Money …)

Deliverables

Shared document with Section 1

Be ready to explain: Why the project exists; scope.

React deliverable

Add a short user journey and one labeled React mockup to the introduction. Describe the app as a local personal tracker with demo data.

AI-agent prompt

Work on #09 (Documentation: create the shared document and write Section 1) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Add a short user journey and one labeled React mockup to the introduction. Describe the app as a local personal tracker with demo data. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#10 Enums and exception hierarchy

Isa Bilalli

Sat 14 Nov Needs #5 2 h v4: T-05 · FR-23 | v5.1 source p. 12

Fixed lists of allowed values and one family of clear errors for the whole system.

Task steps

1. Create the Enums: Category (SALARY, FOOD, RENT, BILLS, TRANSPORT, FUN, HEALTH, EDUCATION, OTHER), Kind (INCOME, EXPENSE, TRANSFER), AccountType (CASH, BANK, CARD, SAVINGS), Frequency (WEEKLY, MONTHLY, YEARLY).

2. Category.from_text('food') is not case sensitive; an unknown value gives a message that lists all valid categories.

3. Frequency.next_date(d) must handle month ends correctly (see the code).

4. Create FinanceError and its subclasses: InvalidAmountError, InvalidDateError, AccountNotFoundError, TransactionNotFoundError, CurrencyMismatchError, BudgetError, DataImportError.

5. Write at least 5 tests.

Done when

- Category.from_text('food') returns Category.FOOD
- Unknown category message lists all valid values
- MONTHLY after 31 Jan 2027 is 28 Feb 2027
- Every custom error is a FinanceError; 5+ tests pass

Deliverables

pft/domain/category.py, pft/domain/exceptions.py, tests

Be ready to explain: Enum, custom exception hierarchy, class methods.

React deliverable

Map API errors to inline field messages and a recoverable page alert. Use readable enum labels for account types and transaction categories.

AI-agent prompt

Work on #10 (Enums and exception hierarchy) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Map API errors to inline field messages and a recoverable page alert. Use readable enum labels for account types and transaction categories. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
def add_month(d: date) -> date:
    y, m = (d.year + 1, 1) if d.month == 12 else (d.year, d.month + 1)
day = min(d.day, calendar.monthrange(y, m)[1]) # 31 Jan -> 28/29 Feb
    return date(y, m, day)
```

#11 Ledger: accounts, events and Observer

Florent Latifi

Sat 14 Nov – Sun 15 Nov Needs #7 3 h v4: T-06 · FR-05 | v5.1 source p. 13

The central service. It holds all accounts and tells other parts (like budgets) when something changes, without knowing them.

Task steps

1. Keep accounts in a private dictionary _accounts. Add open_account(name, type, opening_balance), close_account(id) and account(id) (raises AccountNotFoundError).

2. Add add_transaction(account_id, t) and remove_transaction(account_id, tx_id).

3. Add net_worth as a @property and all_transactions(), a generator that walks all accounts with itertools.chain.

4. Define the Observer Protocol with update(event), and the events as frozen dataclasses: TransactionAdded, TransactionRemoved.

5. Add subscribe, unsubscribe and _notify. The Ledger never prints and never touches files.

6. Write at least 6 tests, using a small test observer that records events.

Done when

- Net worth = sum of all balances
- A test observer gets exactly one event per added transaction
- Unknown account ID raises AccountNotFoundError
- No print() and no file access in ledger.py; 6+ tests pass

Deliverables

pft/services/ledger.py, tests/unit/test_ledger.py

Be ready to explain: Observer pattern, typing.Protocol, composition, itertools.chain.

React deliverable

Show per-account balance and total net worth from API responses. Refresh affected accounts and the dashboard after successful mutations.

AI-agent prompt

Work on #11 (Ledger: accounts, events and Observer) for Florent Latifi. Read this task and its dependencies first. React deliverable: Show per-account balance and total net worth from API responses. Refresh affected accounts and the dashboard after successful mutations. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
class Observer(Protocol):
    def update(self, event: LedgerEvent) -> None: ...
class Ledger:
    def subscribe(self, observer: Observer) -> None:
        self._observers.append(observer)
    def add_transaction(self, account_id: str, t: Transaction) -> None:
        self.account(account_id).add(t)
        self._notify(TransactionAdded(account_id, t))
```

#12 Validators: descriptors and decorators

Isa Bilalli

Sat 14 Nov – Sun 15 Nov Needs #6 #10 2 h v4: T-07 · FR-23 | v5.1 source p. 14

The same validation rule is written once and reused everywhere. This is also one of the strongest “Advanced Python” points for the defense.

Task steps

1. Write the descriptor PositiveAmount with __set_name__, __get__ and __set__; it raises InvalidAmountError for zero or negative Money.
2. Write the descriptor NonEmptyText for names and notes that must not be empty.
3. Write the decorator with arguments @log_action('add') using functools.wraps and the logging module.
4. Write @require_positive for functions that receive an amount.
5. Write at least 5 tests; check the log line with pytest’s caplog fixture.

Done when

- Amount 0 or −5 raises an error with a clear message
- Decorated functions keep __name__ and docstring
- Each decorated action writes exactly one log line
- 5+ tests pass

Deliverables

pft/domain/validators.py, tests/unit/test_validators.py

Be ready to explain: Descriptors, decorators with arguments, functools.wraps, logging.

React deliverable

Validate required fields and positive decimal strings before submit. Display server errors beside matching fields, preserve input and prevent double submission.

AI-agent prompt

Work on #12 (Validators: descriptors and decorators) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Validate required fields and positive decimal strings before submit. Display server errors beside matching fields, preserve input and prevent double submission. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
class PositiveAmount:
    def __set_name__(self, owner, name):
        self.private = "_" + name
    def __get__(self, obj, objtype=None):
        return self if obj is None else getattr(obj, self.private)
    def __set__(self, obj, value: Money):
        if value.amount <= 0:
            raise InvalidAmountError("Amount must be a positive number")
        setattr(obj, self.private, value)
def log_action(name: str):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = func(*args, **kwargs)
            log.info("%s: %s", name, func.__name__)
            return result
        return wrapper
    return decorator
```

#13 Check-in #1

Erti Hoxha, Isa Bilalli, Florent Latifi

Fri 13 Nov Needs #4 0.5 h | v5.1 source p. 14

A short meeting to catch problems early, before the weekend.

Task steps

1. Each member answers three questions in 2 minutes: What did I finish? What do I do next? Am I blocked?

2. The Project Manager moves the cards on the board and, if needed, moves a task to another member.

3. Check that #5 (setup) is integrated on main and that #6 (Money) and #7 (Account) will be merged by Saturday 14 November.

Done when

• Board is up to date

• Every blocker has an owner and a date

Deliverables

Short notes in the chat

React deliverable

Have all three run the frontend, check route placeholders and confirm fixture data. Record setup, accessibility and API contract blockers with owners.

AI-agent prompt

Work on #13 (Check-in #1) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Have all three run the frontend, check route placeholders and confirm fixture data. Record setup, accessibility and API contract blockers with owners. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#14 Transaction hierarchy: Income, Expense, Transfer

Isa Bilalli

Sun 15 Nov – Tue 17 Nov Needs #6 #10 #12 5 h v4: T-08 · FR-02, FR-03 | v5.1 source p. 15

The abstract base class and its three kinds. This is where abstraction, inheritance and polymorphism live.

Task steps

1. Write the abstract dataclass Transaction: id, account_id, amount (Money, validated by PositiveAmount), date, category, note.

2. Add @abstractmethod signed_amount() and a kind property.

3. Income returns +amount, Expense returns −amount, Transfer has direction (OUT / IN) and transfer_id and returns −amount for OUT and +amount for IN.

4. Add to_row() that returns exactly the CSV columns from Appendix B, __lt__ by date (for sorting), and copy_for(new_date) for recurring items.

5. Tell Florent Latifi when it is merged, so the stub in the Account tests can be replaced.

6. Write at least 8 tests.

6. Check keyword-only dataclass fields and descriptor defaults before testing Transfer construction.

Done when

• Transaction(...) directly raises TypeError

• Signed amounts are correct for all three kinds

• Zero or negative amounts are rejected

• to_row() gives exactly the agreed columns; sorted() orders by date; 8+ tests pass

Deliverables

pft/domain/transaction.py, tests/unit/test_transaction.py

Be ready to explain: ABC, inheritance, polymorphism, dataclasses.

React deliverable

Build a shared income/expense form for account, amount, date, category and note. Distinguish income, expense and transfer in rows without relying on color alone.

AI-agent prompt

Work on #14 (Transaction hierarchy: Income, Expense, Transfer) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Build a shared income/expense form for account, amount, date, category and note. Distinguish income, expense and transfer in rows without relying on color alone. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
@dataclass
class Transaction(ABC):
    account_id: str
    amount: Money
    date: date
    category: Category
    note: str = ""
    id: str = field(default_factory=new_id)
    @abstractmethod
    def signed_amount(self) -> Money: ...
class Expense(Transaction):
    def signed_amount(self) -> Money:
        return -self.amount
```

#15 Sorting and text search

Erti Hoxha

Tue 17 Nov Needs #8 1.5 h v4: T-09 · FR-06, FR-07 | v5.1 source p. 15

Users want to see the biggest expenses first or find “lunch” in their notes.

Task steps

1. Write sort_transactions(items, key, reverse) for key 'date', 'amount' or 'category', using operator.attrgetter.
2. Write the filter text_contains(text): not case sensitive, searches the note.
3. An invalid sort key raises ValueError that lists the valid keys.
4. Write at least 4 tests.

Done when

- Sorting is stable and descending order works
- Searching 'lunch' finds 'Lunch with team'
- 4+ tests pass

Deliverables

pft/services/filters.py (additions), tests

Be ready to explain: Key functions, operator module, stable sorting.

React deliverable

Add date, amount and category sorting plus case-insensitive note search. Keep filter and sort state together and test the visible result order.

AI-agent prompt

Work on #15 (Sorting and text search) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Add date, amount and category sorting plus case-insensitive note search. Keep filter and sort state together and test the visible result order. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#16 Exporter base class and CsvExporter

Erti Hoxha

Tue 17 Nov – Wed 18 Nov Needs #5 3 h v4: T-10 · FR-11 | v5.1 source p. 16

One common interface for all export formats. A new format later is just one new class.

Task steps

1. Write the abstract Exporter with a registry that subclasses fill through __init_subclass__(fmt=...).
2. Add the class method Exporter.for_format(fmt) and the abstract method export(rows, path).
3. Write CsvExporter with csv.DictWriter, UTF-8 and a header row.
4. Write format_value() with functools.singledispatch: Money → '12.50', date → ISO text, Enum → its name.
5. An unknown format gives an error that lists the known formats. Write at least 4 tests using tmp_path.

Done when

- for_format('csv') returns a CsvExporter
- The exported file opens in Excel with the right columns
- for_format('xml') gives a clear error
- 4+ tests pass

Deliverables

pft/infrastructure/exporters.py, tests/unit/test_exporters.py

Be ready to explain: Strategy pattern, __init_subclass__, singledispatch.

React deliverable

Add a CSV download control for transactions and reports. Pass the current filters and filename, and show pending and failed download states.

AI-agent prompt

Work on #16 (Exporter base class and CsvExporter) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Add a CSV download control for transactions and reports. Pass the current filters and filename, and show pending and failed download states. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
class Exporter(ABC):
    registry: ClassVar[dict[str, type["Exporter"]]] = {}
    def __init_subclass__(cls, fmt: str = "", **kwargs) -> None:
        super().__init_subclass__(**kwargs)
        if fmt:
Exporter.registry[fmt] = cls # self-registration
    @classmethod
    def for_format(cls, fmt: str) -> "Exporter":
        try:
            return cls.registry[fmt]()
        except KeyError:
            known = ", ".join(sorted(cls.registry))
            raise ValueError(f"Unknown format '{fmt}'. Known: {known}") from None
class CsvExporter(Exporter, fmt="csv"): ...
```

#17 TransactionFactory

Isa Bilalli

Tue 17 Nov – Wed 18 Nov Needs #14 2 h v4: T-11 · FR-02, FR-03 | v5.1 source p. 16

One single place that turns user text or a CSV row into the right Transaction object.

Task steps

1. Write from_input(kind, amount, category, date, account_id, note) that parses and validates text from the CLI.
2. Write from_row(row) that builds an object from a CSV row.
3. Use a dictionary {Kind.INCOME: Income, ...} instead of long if/elif chains.
4. Accept '12,50' as 12.50. Invalid data raises the right FinanceError subclass.
5. Write at least 6 tests, including a round trip for all three kinds.

Done when

- from_row(t.to_row()) == t for all three kinds
- '12,50' is accepted; 'abc' raises InvalidAmountError
- A wrong date raises InvalidDateError
- 6+ tests pass

Deliverables

pft/domain/factory.py, tests/unit/test_factory.py

Be ready to explain: Factory pattern, dictionary dispatch.

React deliverable

Keep transaction DTO mapping in a shared API adapter. Map API kind/direction to form values and row labels, and cover unknown types with a readable error.

AI-agent prompt

Work on #17 (TransactionFactory) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Keep transaction DTO mapping in a shared API adapter. Map API kind/direction to form values and row labels, and cover unknown types with a readable error. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#18 Repository and atomic save

Florent Latifi

Tue 17 Nov – Wed 18 Nov Needs #11 #17 5 h v4: T-12 · FR-13, NFR-02 | v5.1 source p. 17

Saves and loads all data and makes sure a file is never left half-written, even if the program stops.

Task steps

1. Write the abstract Repository with load() → Ledger and save(ledger).

2. Write CsvRepository: accounts.csv, transactions.csv, budgets.csv, recurring.csv in data/ (columns in Appendix B). Use the factory from #17 to read rows.

3. Write JsonRepository: the same data in one JSON file.

4. Write the context manager atomic_write(path) and use it for every save.

5. First start without a data/ folder gives an empty Ledger, not an error.

6. Write at least 6 tests; run the same tests for CSV and JSON with @pytest.mark.parametrize.

6. Extend the SQLite setup from #05 with SqliteRepository behind the Repository interface. Persist exact amounts as integer cents or decimal text and test rollback. Confirm any course-specific CSV requirements.

Done when

• Save load gives the same data (round trip)

• A simulated crash during writing leaves the old file unchanged

• Same tests pass for CSV and JSON

• 6+ tests pass

Deliverables

pft/infrastructure/repository.py, tests/integration/test_repository.py

Be ready to explain: Repository pattern, context managers, pathlib, csv, json.

React deliverable

Show saving, saved and failed states from the API. Verify transactions and balances remain correct after refresh and backend restart. Keep SQLite access in Python.

AI-agent prompt

Work on #18 (Repository and atomic save) for Florent Latifi. Read this task and its dependencies first. React deliverable: Show saving, saved and failed states from the API. Verify transactions and balances remain correct after refresh and backend restart. Keep SQLite access in Python. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
@contextmanager
def atomic_write(path: Path) -> Iterator[TextIO]:
    tmp = path.with_suffix(path.suffix + ".tmp")
    try:
        with tmp.open("w", newline="", encoding="utf-8") as f:
            yield f
os.replace(tmp, path) # old file replaced only on success
    finally:
        if tmp.exists():
tmp.unlink() # clean up after a crash
```

#19 Report base class and MonthlySummary

Florent Latifi

Tue 17 Nov – Fri 20 Nov Needs #8 #11 4 h v4: T-13 · FR-08 | v5.1 source p. 18

The base for every report (Template Method) and the most important report: the monthly summary.

Task steps

1. Write the abstract Report: build() calls collect() and then summarize(); the result is a ReportData dataclass.

2. Write MonthlySummary(year, month): income, expenses, net, savings rate %, top 3 spending categories, difference to the previous month.

3. Ignore transfers. Use the filters from #8 and itertools / collections.Counter.

4. A month with no data shows zeros, not an error.

5. Write at least 5 tests.

Done when

- Income 800 and expenses 600 net 200.00, savings rate 25.0%
- Empty month shows zeros
- Transfers do not change the result
- 5+ tests pass

Deliverables

pft/services/reports.py, tests/unit/test_reports.py

Be ready to explain: Template Method pattern, ABC, itertools.

React deliverable

Build the Dashboard month selector, totals, savings rate, top categories and previous-month comparison from API data. Add loading, empty and retry states.

AI-agent prompt

Work on #19 (Report base class and MonthlySummary) for Florent Latifi. Read this task and its dependencies first. React deliverable: Build the Dashboard month selector, totals, savings rate, top categories and previous-month comparison from API data. Add loading, empty and retry states. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
class Report(ABC):
def build(self) -> ReportData: # template method: fixed steps
        rows = self.collect()
        return self.summarize(rows)
    @abstractmethod
    def collect(self) -> Iterable[Transaction]: ...
    @abstractmethod
    def summarize(self, rows: Iterable[Transaction]) -> ReportData: ...
```

#20 Documentation: Section 2, first draft

Erti Hoxha

Wed 18 Nov – Sat 21 Nov Needs #4 #14 #19 3 h v4: D-02 (part 1) | v5.1 source p. 18

Section 2 explains how the system is designed and why. The design is fixed since the kickoff, so the draft can be written now.

Task steps

1. 2.1 Requirements: a short table of the main functional requirements (Appendix A) and the most important non-functional ones (Decimal money, atomic save, 80% coverage).

2. 2.2 Architecture: describe the four layers (presentation, service, domain, infrastructure) and dependency direction toward the domain and stable interfaces. Insert the layer diagram as Figure 1.

3. 2.3 Domain model: describe Money, Transaction and its subclasses, Account, Ledger, Budget, RecurringTransaction. Insert a draft UML class diagram as Figure 2.

4. 2.4 Design patterns: one short paragraph each for Repository, Strategy, Template Method, Observer, Factory and Value Object — where and why.

5. 2.5 Data model: the CSV files and how a transfer is stored as two linked rows.

6. Ask Isa Bilalli and Florent Latifi to check that the names match the real code.

Done when

- All five sub-parts exist in draft
- Figures 1 and 2 are placed and mentioned in the text (“as shown in Figure 1”)
- Names match the code

Deliverables

Section 2 draft in the shared document

Be ready to explain: Architecture and design patterns.

React deliverable

Draft a React-to-FastAPI request diagram and screen-to-requirement map. Document where validation, persistence and rendering happen.

AI-agent prompt

Work on #20 (Documentation: Section 2, first draft) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Draft a React-to-FastAPI request diagram and screen-to-requirement map. Document where validation, persistence and rendering happen. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#21 First CLI commands

Florent Latifi

Wed 18 Nov – Fri 20 Nov Needs #11 #18 #19 3 h v4: T-14 · FR-22, FR-23 | v5.1 source p. 19

The first working command line, so the whole flow can be used end to end for milestone W1.

Task steps

1. Use argparse sub-commands: account add/list, add income/expense, list (with filter options), balance, report monthly.
2. Dispatch with match / case. Load the data at start and save after every change.
3. Catch every FinanceError and show a friendly message (exit code 1). Never show a Python traceback.
4. Make sure --help works for every command.
5. Write at least 4 CLI tests with pytest's capsys.

Done when

- pft add expense 12.50 FOOD --account cash works and the data is still there after a restart
- Invalid input never shows a traceback
- --help works everywhere
- 4+ tests pass

Deliverables

pft/cli.py, pft/__main__.py, tests/integration/test_cli.py

Be ready to explain: argparse, structural pattern matching, layered architecture.

React deliverable

Implement the shared fetch client and connect account, transaction and monthly-summary routes to FastAPI. Verify backend errors and retain course-required CLI behavior until a substitution is approved.

AI-agent prompt

Work on #21 (First CLI commands) for Florent Latifi. Read this task and its dependencies first. React deliverable: Implement the shared fetch client and connect account, transaction and monthly-summary routes to FastAPI. Verify backend errors and retain course-required CLI behavior until a substitution is approved. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
match args.command:
    case "add":
        app.add(args.kind, Money(args.amount), Category.from_text(args.category))
    case "report":
        app.report(args.type, args.month)
    case _:
        parser.print_help()
```

#22 Milestone W1 review: the core works

Erti Hoxha, Isa Bilalli, Florent Latifi

Fri 20 Nov Needs #18 #21 1 h | v5.1 source p. 19

We check that the full flow works before we add advanced features, and we decide on the optional features.

Task steps

1. Run the full flow live: create an account → add income and expense → close the program → open it again → monthly report.

2. Check the numbers: tests are green, CI is green.

3. Each member says how many hours are left on their list.

4. Decide on the Could features (#28): who is ahead of plan takes one of undo/redo, bank CSV import or charts. If nobody is ahead, we skip them.

5. Check that Section 1 is done and Section 2 has a draft.

Done when

• add save load monthly summary works from the CLI

• Decision on Could features is written in the chat

Deliverables

Meeting notes

React deliverable

Run the core web flow with demo data: create account, add income and expense, refresh and confirm the dashboard totals. Record evidence and defer optional UI work if the core is unstable.

AI-agent prompt

Work on #22 (Milestone W1 review: the core works) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Run the core web flow with demo data: create account, add income and expense, refresh and confirm the dashboard totals. Record evidence and defer optional UI work if the core is unstable. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#23 Transfers between accounts

Isa Bilalli

Sat 21 Nov Needs #11 #14 2 h v4: T-15 · FR-14 | v5.1 source p. 20

Moving money between two accounts. It is not income or expense, so it must never change the reports.

Task steps

1. Add Ledger.transfer(src, dst, money, date, note). Coordinate with Florent Latifi, who owns ledger.py.
2. It creates two linked Transfer objects with the same transfer_id: direction OUT on the source, IN on the target.
3. Both rows are added or none (if the second fails, remove the first).
4. Deleting one side deletes both. Reports ignore transfers.
5. Reject a transfer to the same account and a transfer that would make the source balance negative.
6. Write at least 5 tests.

Done when

- Bank 100.00 Cash 40.00 gives Bank 60.00, Cash 40.00, net worth unchanged
- Same-account transfer is rejected
- Transfer above the balance is rejected
- 5+ tests pass

Deliverables

pft/services/ledger.py, pft/domain/transaction.py, tests

Be ready to explain: All-or-nothing operations, linked objects.

React deliverable

Add a Transfer form with distinct source and destination accounts, amount and date. Refresh both balances after success and keep transfers out of income/expense totals.

AI-agent prompt

Work on #23 (Transfers between accounts) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Add a Transfer form with distinct source and destination accounts, amount and date. Refresh both balances after success and keep transfers out of income/expense totals. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#24 JSON and Markdown exporters + CSV injection guard

Erti Hoxha

Sat 21 Nov – Sun 22 Nov Needs #16 3 h v4: T-17 · FR-12, NFR-08 | v5.1 source p. 20

Two more formats added without changing any existing class (Open/Closed), and protection for CSV files opened in Excel.

Task steps

1. Write JsonExporter (indent 2, amounts as text) and MarkdownExporter (a table). Each registers itself with its format name.

2. Add the CSV injection guard: text values that start with =, +, - or @ get a leading apostrophe ("=SUM(A1)" → "'=SUM(A1)").

3. Do not change CsvExporter or the Exporter base class for the new formats.

4. Write at least 5 tests.

Done when

• for_format('json') and for_format('md') work; the JSON file loads with json.load

• The Markdown file shows as a table

• "=SUM(A1)" is exported as "'=SUM(A1)"

• No existing class changed; 5+ tests pass

Deliverables

pft/infrastructure/exporters.py, tests

Be ready to explain: Strategy pattern, Open/Closed principle, security.

React deliverable

Add the supported export format picker. Verify downloads match active filters, test CSV formula-neutralization via the backend, and display only approved formats.

AI-agent prompt

Work on #24 (JSON and Markdown exporters + CSV injection guard) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Add the supported export format picker. Verify downloads match active filters, test CSV formula-neutralization via the backend, and display only approved formats. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#25 Recurring transactions

Isa Bilalli

Sat 21 Nov – Sun 22 Nov Needs #14 #17 4 h v4: T-16 · FR-18 | v5.1 source p. 21

Rent, salary and subscriptions are created automatically, so the user does not type them every month.

Task steps

1. Write RecurringTransaction: template transaction, frequency, start, optional end, last_run.

2. Write the generator occurrences(until) and apply_due(ledger, today), which adds only the items that are missing.

3. Day 31 in a short month moves to the last day of that month (use the next_date logic from #10).

4. Save last_run, so a restart never creates duplicates.

5. Write at least 6 tests.

6. Derive each monthly occurrence from the original start date plus its occurrence index. Test 31 January, 28 February and 31 March.

Done when

- Monthly rent from 1 September, today 15 November → exactly 3 items
- Running apply_due twice creates no duplicates (also after restart)
- The end date is respected
- 6+ tests pass

Deliverables

pft/domain/recurring.py, tests/unit/test_recurring.py

Be ready to explain: Generators and yield, date logic.

React deliverable

Add recurrence frequency and start/end fields to the transaction form. Show upcoming occurrences and verify repeated loads do not create duplicates or shift month-end rules.

AI-agent prompt

Work on #25 (Recurring transactions) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Add recurrence frequency and start/end fields to the transaction form. Show upcoming occurrences and verify repeated loads do not create duplicates or shift month-end rules. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Original interface sketch

```
def occurrences(self, until: date) -> Iterator[Transaction]:
    day = self.next_due
    while day <= until and (self.end is None or day <= self.end):
        yield self.template.copy_for(day)
        day = self.frequency.next_date(day)
```

#26 CategoryReport

Florent Latifi

Sun 22 Nov Needs #19 2 h v4: T-19 · FR-09 | v5.1 source p. 21

Shows where the money goes: spending per category for any period.

Task steps

1. Subclass Report; implement only collect() and summarize().
2. For each category: amount in EUR and % of total spending.
3. Sort from highest to lowest; hide categories with 0.
4. Write at least 4 tests.

Done when

- Percentages add up to 100.0 (±0.1)
- Works for one day and for a full year
- 4+ tests pass

Deliverables

pft/services/reports.py, tests

Be ready to explain: Inheritance, Template Method.

React deliverable

Add the category breakdown to Dashboard with a date range, EUR amounts, percentages and a chart plus accessible table. Verify empty periods and totals.

AI-agent prompt

Work on #26 (CategoryReport) for Florent Latifi. Read this task and its dependencies first. React deliverable: Add the category breakdown to Dashboard with a date range, EUR amounts, percentages and a chart plus accessible table. Verify empty periods and totals. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#27 Budgets and BudgetMonitor (Observer)

Isa Bilalli

Sun 22 Nov – Tue 24 Nov Needs #11 #14 5 h v4: T-18 · FR-15, FR-16 | v5.1 source p. 22

Monthly budgets per category, with automatic warnings, without the Ledger knowing anything about budgets.

Task steps

1. Write Budget(category, month, limit) with usage(ledger) in % and is_exceeded().

2. Write BudgetMonitor, which implements the Observer protocol and reacts to TransactionAdded for expenses only.

3. Send warnings at 80% and 100%, only once per level per month. Warnings go to a callback; the CLI prints them.

4. Use constants (BUDGET_WARNING = Decimal('0.80')), not magic numbers.

5. Write at least 6 tests.

Done when

• FOOD budget 200.00, spent 150.00, new expense 15.00 → “FOOD budget at 82%”

• Income never creates a warning

• A second expense above 80% does not repeat the 80% warning

• 6+ tests pass

Deliverables

pft/domain/budget.py, pft/services/budget_monitor.py, tests

Be ready to explain: Observer pattern, separation of concerns.

React deliverable

Build Budgets with category/month inputs and progress. Show the 80% and 100% thresholds with text, update usage after spending and render warnings from the API.

AI-agent prompt

Work on #27 (Budgets and BudgetMonitor (Observer)) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Build Budgets with category/month inputs and progress. Show the 80% and 100% thresholds with text, update usage after spending and render warnings from the API. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#28 Optional (Could) features, only if decided at #22

Erti Hoxha (scope coordinator)

Sun 22 Nov – Fri 27 Nov Needs #22 — v4: T-23 · FR-19, FR-20, FR-21 | v5.1 source p. 22

Extra points of “advanced” work, but only after all Must and Should tasks of that member are done.

Task steps

1. Undo / redo with the Command pattern: AddCommand, DeleteCommand, TransferCommand, each with execute() and undo(); two stacks for history.
2. BankCsvImporter: valid rows are imported; invalid rows are listed with row number and reason.
3. Charts with matplotlib: a category pie chart and a monthly trend line, saved as PNG.
4. Each feature has its own tests and can be removed without breaking anything.

Done when

- Started only after the member’s Must/Should tasks are done
- Own tests pass
- Removing the feature breaks nothing

Deliverables

pft/services/commands.py, pft/infrastructure/importer.py, pft/infrastructure/charts.py

Be ready to explain: Command pattern, validation, visualisation.

React deliverable

Gate optional React additions behind explicit team approval: CSV import preview, undo/redo controls or chart PNG download. For any selected feature, add a UI acceptance check and keep it off the core path.

AI-agent prompt

Work on #28 (Optional (Could) features, only if decided at #22) for Erti Hoxha (scope coordinator). Read this task and its dependencies first. React deliverable: Gate optional React additions behind explicit team approval: CSV import preview, undo/redo controls or chart PNG download. For any selected feature, add a UI acceptance check and keep it off the core path. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#29 Documentation input for Section 3: the domain

Isa Bilalli

Tue 24 Nov Needs #6 #14 #27 2 h v4: D-09 | v5.1 source p. 22

Erti Hoxha writes Section 3 but needs the facts from the person who wrote the code.

Task steps

1. Write about half a page: how Money, the Transaction hierarchy, the validators and the BudgetMonitor work.

2. Choose 2–3 short code snippets (under 15 lines each): Money, Transaction + Expense, BudgetMonitor.update().

3. List your tests: file name, number of tests, what they check.

4. Put everything in the shared folder and tell Erti Hoxha.

Done when

• Delivered to Erti Hoxha by Tuesday 24 November

• Snippets match the code in main

Deliverables

Text + snippets in the shared folder

React deliverable

Document how React sends Money values and transaction data to domain operations. Include one short request example and identify validation on each side.

AI-agent prompt

Work on #29 (Documentation input for Section 3: the domain) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Document how React sends Money values and transaction data to domain operations. Include one short request example and identify validation on each side. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#30 Code walk-through

Erti Hoxha, Isa Bilalli, Florent Latifi

Tue 24 Nov Needs #22 1.5 h | v5.1 source p. 23

Defense training. Each member explains their code to the others, so everyone understands the whole system.

Task steps

1. Each member has 25 minutes: show your classes on screen, explain the OOP and Python features in them, and answer questions.

2. The others write down one question they could get in the defense, and ask it.

3. The Project Manager notes every part that someone could not explain; that member studies it again.

Done when

• Each member presented

• List of weak points is in the chat

Deliverables

Notes

React deliverable

Trace one React action through FastAPI, Ledger and SQLite, then back to the screen. Have each member explain a route, error case and test.

AI-agent prompt

Work on #30 (Code walk-through) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Trace one React action through FastAPI, Ledger and SQLite, then back to the screen. Have each member explain a route, error case and test. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#31 TrendReport and BudgetReport

Isa Bilalli

Tue 24 Nov – Wed 25 Nov Needs #19 #27 3 h v4: T-20 · FR-10, FR-17 | v5.1 source p. 23

Two more reports on Florent Latifi’s base class; they show code reuse through the Template Method.

Task steps

1. TrendReport(last N months): income, expenses and net per month; months without data show 0.
2. BudgetReport(month): planned, actual and difference per category, with status OK / WARNING / OVER.
3. Implement only collect() and summarize(); never copy build().
4. Write at least 5 tests.

Done when

- N = 6 gives exactly 6 rows in date order
- Statuses match the 80% and 100% limits
- build() is not copied
- 5+ tests pass

Deliverables

pft/services/reports.py, tests

Be ready to explain: Template Method, code reuse.

React deliverable

Add planned-versus-actual budget reporting to the Budgets screen. Include trend visualization only if retained, and map month filters and empty results.

AI-agent prompt

Work on #31 (TrendReport and BudgetReport) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Add planned-versus-actual budget reporting to the Budgets screen. Include trend visualization only if retained, and map month filters and empty results. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#32 Export commands in the CLI

Erti Hoxha

Tue 24 Nov – Wed 25 Nov Needs #21 #24 1.5 h v4: T-21 · FR-11, FR-12 | v5.1 source p. 23

Makes all export formats available from the command line. Coordinate with Florent Latifi, who owns cli.py.

Task steps

1. Add pft export transactions --format csv|json|md --out FILE.
2. Add pft report monthly 2026-11 --export csv.
3. Use only Exporter.for_format(): no format names written in the CLI code.
4. Write at least 3 tests.

Done when

- Each format creates the file and prints its path
- Unknown format gives a friendly error with the list of formats
- 3+ tests pass

Deliverables

pft/cli.py, tests

Be ready to explain: Loose coupling between layers.

React deliverable

Wire React export controls to the approved download endpoints. Test CSV and JSON with current filters and verify content type and failure messages.

AI-agent prompt

Work on #32 (Export commands in the CLI) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Wire React export controls to the approved download endpoints. Test CSV and JSON with current filters and verify content type and failure messages. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#33 Documentation: finish Section 2

Erti Hoxha

Wed 25 Nov Needs #20 2 h v4: D-02 (part 2) | v5.1 source p. 24

The features are almost final now, so Section 2 can be completed and checked against the code.

Task steps

1. Add the add-expense flow (user → CLI → factory → ledger → budget monitor → repository) as Figure 3.
2. Add the reports / exporters / repository diagram as Figure 4.
3. Update every class name and method name to the real code.
4. Read Section 2 with Isa Bilalli and Florent Latifi for 15 minutes each and fix what they find.
5. Target length: about 2.5 pages.

Done when

- Figures 1–4 placed, numbered and mentioned in the text
- Isa Bilalli and Florent Latifi approved the text for their parts
- About 2.5 pages

Deliverables

Section 2 in the shared document

React deliverable

Finalize the architecture section with routes, DTOs, request flow and desktop/mobile layout. Link each frontend feature to its requirement and task ID.

AI-agent prompt

Work on #33 (Documentation: finish Section 2) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Finalize the architecture section with routes, DTOs, request flow and desktop/mobile layout. Link each frontend feature to its requirement and task ID. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#34 Documentation input for Section 3: storage, reports, CLI

Florent Latifi

Wed 25 Nov Needs #18 #19 #21 3 h v4: D-10 | v5.1 source p. 24

Erti Hoxha needs real screenshots and real results for the Implementation section.

Task steps

1. Write about half a page on the Repository, atomic save, reports and the CLI.
2. Take CLI screenshots: adding an expense, the monthly report, a budget warning, an export.
3. Run pytest --cov and take a screenshot of the coverage report.
4. Put everything in the shared folder and tell Erti Hoxha.

Done when

• Delivered to Erti Hoxha by Wednesday 25 November

• Screenshots are from the newest version

Deliverables

Text + screenshots in the shared folder

React deliverable

Provide implementation notes for SQLite persistence, FastAPI routes, the React API client and loading/error handling. Include one real integration screenshot.

AI-agent prompt

Work on #34 (Documentation input for Section 3: storage, reports, CLI) for Florent Latifi. Read this task and its dependencies first. React deliverable: Provide implementation notes for SQLite persistence, FastAPI routes, the React API client and loading/error handling. Include one real integration screenshot. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#35 Check-in #2

Erti Hoxha, Isa Bilalli, Florent Latifi

Wed 25 Nov Needs #22 0.5 h | v5.1 source p. 24

Last check before the code freeze on Friday 27 November.

Task steps

1. Each member lists what is still open before the freeze.
2. Check the test count and the coverage.
3. Decide what to cut if something will not be ready (first Could, then Should, never Must).

Done when

• A clear list of what must be merged by Saturday

Deliverables

Notes

React deliverable

Review all four screens and decide which defects must be fixed before freeze. Assign each blocker and list any optional UI scope deferred.

AI-agent prompt

Work on #35 (Check-in #2) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Review all four screens and decide which defects must be fixed before freeze. Assign each blocker and list any optional UI scope deferred. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#36 Full CLI menu and integration

Florent Latifi

Wed 25 Nov – Fri 27 Nov Needs #21 #23 #24 #25 #26 #27 #31 #32 5 h | v5.1 source p. 25

v4: T-22 · FR-22, NFR-10 Why: Connects everything into one finished application.

Task steps

1. Add the interactive menu (pft menu) with numbered options for every feature.
2. At start: apply due recurring transactions and subscribe the BudgetMonitor to the Ledger.
3. Log every change to logs/pft.log with the logging module.
4. Write end-to-end tests for the main flows and fix integration bugs.
5. Run the manual test checklist (every command once).

Done when

- Every Must and Should requirement can be used from the menu
- The manual test checklist passes
- The log shows every change; CI is green

Deliverables

pft/cli.py, tests/integration/, manual test checklist

Be ready to explain: Layered architecture, integration.

React deliverable

Integrate Dashboard, Transactions, Budgets and Accounts in the production React build. Verify navigation, CRUD, transfers, budget updates, export and backend restart with synthetic data.

AI-agent prompt

Work on #36 (Full CLI menu and integration) for Florent Latifi. Read this task and its dependencies first. React deliverable: Integrate Dashboard, Transactions, Budgets and Accounts in the production React build. Verify navigation, CRUD, transfers, budget updates, export and backend restart with synthetic data. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#37 Finish tests: 60+ tests and 80% coverage

Erti Hoxha, Isa Bilalli, Florent Latifi

Sat 21 Nov – Fri 27 Nov Needs — Erti 4 · Isa 5 · Florent 6 v4: T-24 · NFR-03, NFR-05 | v5.1 source p. 25

Quality goals from the requirements. Each member tests their own code; this task is the final push before the freeze.

Task steps

1. Run pytest --cov and look at the lines that are not covered in your own modules.

2. Add tests for the edge cases: amount 0, negative, 'abc', 31 February, empty file, broken CSV row, 29 February.

3. Use the shared fixtures in conftest.py (a sample ledger) and @pytest.mark.parametrize for many inputs.

4. Florent Latifi: add the performance test: monthly report on 10,000 generated transactions in under 1 second.

Done when

• At least 60 tests in total

• Coverage ≥ 80% in total and in each member’s modules

• All tests pass in CI

Deliverables

tests/, coverage report

Be ready to explain: Testing, fixtures, parametrize, what coverage does and does not prove.

React deliverable

Add frontend checks for forms, filters and error states, plus a browser smoke flow across four screens. Run React lint/build alongside the Python test suite.

AI-agent prompt

Work on #37 (Finish tests: 60+ tests and 80% coverage) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Add frontend checks for forms, filters and error states, plus a browser smoke flow across four screens. Run React lint/build alongside the Python test suite. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#38 Slide content: slides 4 and 5

Isa Bilalli

Fri 27 Nov Needs #4 1 h v4: D-07 (input) | v5.1 source p. 25

Erti Hoxha builds the deck, but each speaker provides the content of their own slides.

Task steps

1. Slide 4: domain UML (simplified) + the four OOP principles, each with one class from our code.

2. Slide 5: design patterns: Observer, Strategy, Template Method, Repository (and Command if built), one line each.

3. Write at most 30 words per slide and your speaker notes (40 seconds and 30 seconds).

Done when

• Content sent to Erti Hoxha by Friday 27 November

Deliverables

Slide text + notes

React deliverable

Prepare presentation evidence showing how a React action exercises Python OOP, validation and design patterns. Use a short flow diagram and one code reference.

AI-agent prompt

Work on #38 (Slide content: slides 4 and 5) for Isa Bilalli. Read this task and its dependencies first. React deliverable: Prepare presentation evidence showing how a React action exercises Python OOP, validation and design patterns. Use a short flow diagram and one code reference. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#39 Slide content: slides 6, 7 and 8

Florent Latifi

Fri 27 Nov Needs #4 1 h v4: D-07 (input) | v5.1 source p. 26

Florent Latifi presents the result the user gets, the quality and the demo.

Task steps

1. Slide 6: reports, with a screenshot of the monthly summary.
2. Slide 7: export/import + quality: number of tests and coverage.
3. Slide 8: short demo plan and conclusion.
4. Write at most 30 words per slide and your speaker notes.

Done when

• Content sent to Erti Hoxha by Friday 27 November

Deliverables

Slide text + screenshots + notes

React deliverable

Prepare a React/API integration slide and one screenshot showing a persisted transaction or updated report. Explain how to recover from a failed request.

AI-agent prompt

Work on #39 (Slide content: slides 6, 7 and 8) for Florent Latifi. Read this task and its dependencies first. React deliverable: Prepare a React/API integration slide and one screenshot showing a persisted transaction or updated report. Explain how to recover from a failed request. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#40 CODE FREEZE

Erti Hoxha, Isa Bilalli, Florent Latifi

Fri 27 Nov Needs #36 #37 0.5 h v4: Milestone CF | v5.1 source p. 26

After this point only bug fixes are allowed. The documentation and the slides describe this exact version.

Task steps

1. Review and integrate the final changes on main before the freeze.

2. Verify approved Must and Should features, 60+ Python tests, at least 80% coverage and green Python/React CI.

3. Erti Hoxha tags the verified main commit v1.0-freeze.

4. After freeze, coordinate reviewed bug fixes on main and record the tested commit.

Done when

• The tested main commit has the v1.0-freeze tag.

• Python and React checks pass.

• The frozen demo uses synthetic data.

Deliverables

Git tag

React deliverable

Run Python and React checks on main, freeze the demo dataset and tag the tested commit. Capture a screenshot from the frozen React build.

AI-agent prompt

Work on #40 (CODE FREEZE) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Run Python and React checks on main, freeze the demo dataset and tag the tested commit. Capture a screenshot from the frozen React build. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#41 Final figures and UML diagrams

Erti Hoxha

Sat 28 Nov Needs #40 4 h v4: D-03 | v5.1 source p. 26

Clean, consistent figures for the document and the slides, matching the frozen code.

Task steps

1. Redraw Figures 1–4 in one style (draw.io or PlantUML): same font, same colours.
2. Check every class and method name against the code after the freeze.
3. Prepare Figures 5–8 from the screenshots of #34: CLI, monthly report, chart (if built), coverage report.
4. Number and caption every figure (“Figure 3. Flow of adding an expense”). Text in figures not smaller than 8 pt.

Done when

- At least 6 figures, all numbered and captioned
- Names match the frozen code
- Readable when printed

Deliverables

Figure files (PNG/SVG) placed in the document

React deliverable

Produce labeled screenshots of the four finished screens and one React/API diagram. Use synthetic values, legible labels and useful figure captions.

AI-agent prompt

Work on #41 (Final figures and UML diagrams) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Produce labeled screenshots of the four finished screens and one React/API diagram. Use synthetic values, legible labels and useful figure captions. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#42 Documentation: Section 3 Implementation

Erti Hoxha

Sat 28 Nov – Sun 29 Nov Needs #29 #34 #41 3 h v4: D-04 | v5.1 source p. 27

Shows how the design became code: the most technical section, written from the input of Isa Bilalli and Florent Latifi.

Task steps

1. 3.1 Project structure: the folder tree with one line per module.

2. 3.2 Key code: 3–4 short snippets (Money, BudgetMonitor/Observer, Exporter registry, atomic_write), each explained in 2–3 sentences.

3. 3.3 Advanced Python features: a table “feature → where we use it” (dataclasses, descriptors, decorators, generators, context managers, Protocol, match/case …).

4. 3.4 The application in use: CLI screenshots and report output (Figures 5–7).

5. 3.5 Testing: number of tests, the coverage screenshot (Figure 8), the performance result.

6. Send the section to Isa Bilalli and Florent Latifi for approval of their parts. Target: about 3 pages.

Done when

• Snippets under 15 lines and explained in the text

• Includes test and coverage results

• Approved by Isa Bilalli and Florent Latifi

Deliverables

Section 3 in the shared document

React deliverable

Write the React implementation subsection: routes, shared components, fetch client, form states and API integration tests. Reference actual screenshots and code.

AI-agent prompt

Work on #42 (Documentation: Section 3 Implementation) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Write the React implementation subsection: routes, shared components, fetch client, form states and API integration tests. Reference actual screenshots and code. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#43 Documentation: Section 4 Conclusion and Section 5 References

Erti Hoxha

Sun 29 Nov Needs #9 #33 #42 3 h v4: D-05 | v5.1 source p. 27

Closes the document and lists every source we used.

Task steps

1. Section 4 (about half a page): what we built; what we learned about OOP and advanced Python; the limits (one user, one currency, CLI only); future work (GUI, database, several users).

2. Section 5: every source in IEEE style, numbered in the order of first citation: [1], [2], ...

3. Check both directions: every [n] in the text is in the list, and every source in the list is cited in the text.

4. Use only sources we really used (Python documentation, Design Patterns by Gamma et al., Fluent Python, PEP 8, pytest documentation, OWASP CSV Injection).

Done when

- Conclusion about half a page ↔
- Every citation reference matches
- One reference style (IEEE)

Deliverables

Sections 4 and 5 in the shared document

React deliverable

Summarize what the React interface enables and its limits. Cite only sources used and distinguish local demonstration from future hosted access.

AI-agent prompt

Work on #43 (Documentation: Section 4 Conclusion and Section 5 References) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Summarize what the React interface enables and its limits. Cite only sources used and distinguish local demonstration from future hosted access. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#44 Demo video and backup screenshots

Erti Hoxha

Sat 28 Nov Needs #36 #40 1 h v4: D-08 | v5.1 source p. 27

A backup in case the live demo fails on 8 December.

Task steps

1. Record a 60–90 second screen video of the main flow: add income and expense, monthly report, budget warning, export.

2. Use the frozen version and a clean sample dataset.

3. Save the video and screenshots on the presentation laptop and on a USB stick.

4. Test that it plays offline.

Done when

• Video plays offline on the presentation laptop

• Shows the frozen version

Deliverables

Video file + screenshots

React deliverable

Record a 60–90 second React walkthrough: add a transaction, view dashboard totals, check a budget and export. Keep local backup screenshots for the demo.

AI-agent prompt

Work on #44 (Demo video and backup screenshots) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Record a 60–90 second React walkthrough: add a transaction, view dashboard totals, check a budget and export. Keep local backup screenshots for the demo. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#45 Slides: build the full deck

Erti Hoxha

Wed 25 Nov – Sun 29 Nov Needs #38 #39 4 h v4: D-07 | v5.1 source p. 28

One clear, consistent deck for a 5-minute presentation.

Task steps

1. Build 8 slides with one template: 1 title + problem, 2 features, 3 architecture, 4 domain + OOP, 5 patterns, 6 reports, 7 export + quality, 8 demo + conclusion.

2. Erti Hoxha speaks slides 1–3, Isa Bilalli slides 4–5, Florent Latifi slides 6–8.

3. At most about 30 words per slide; code font at least 18 pt; same fonts and colours everywhere.

4. Add one linking sentence for each handover (“Now Florent Latifi will show what the user gets out of the system.”).

Done when

• 8 slides, same design ≥ ≤

• Code 18 pt, 30 words per slide

• No spelling mistakes

Deliverables

Slide deck (PDF + source file)

React deliverable

Include the four-screen React map and two readable GUI screenshots in the presentation. Connect the demo story to the Python features being assessed.

AI-agent prompt

Work on #45 (Slides: build the full deck) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Include the four-screen React map and two readable GUI screenshots in the presentation. Connect the demo story to the Python features being assessed. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#46 Cross proofreading

Erti Hoxha, Isa Bilalli, Florent Latifi

Sun 29 Nov Needs #43 #45 1 h | v5.1 source p. 28

Spelling, grammar or punctuation mistakes mean an automatic negative evaluation. A second reader finds what the writer cannot see.

Task steps

1. Isa Bilalli reads the whole document slowly, out loud if possible.
2. Florent Latifi reads all slides.
3. Both use comments, not direct edits, so Erti Hoxha sees every change.
4. Check: punctuation at the end of every sentence and list item; same terms everywhere; same number and date format.

Done when

- Document read completely by Isa Bilalli
- Slides read completely by Florent Latifi
- All comments sent to Erti Hoxha

Deliverables

Comments in the document and the slides

React deliverable

Proofread visible React copy, money labels, figure captions and navigation. Check for mismatches between screenshots, documented steps and the running app.

AI-agent prompt

Work on #46 (Cross proofreading) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Proofread visible React copy, money labels, figure captions and navigation. Check for mismatches between screenshots, documented steps and the running app. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#47 Documentation: formatting and final proofreading

Erti Hoxha

Mon 30 Nov Needs #46 3 h v4: D-06 | v5.1 source p. 28

The document must meet every formal rule of the professor.

Task steps

1. Apply all comments from #46.
2. Check Times New Roman 10 pt everywhere: body, headings, tables, captions (code font only if the professor allows it).
3. Check exactly the five required sections, in the right order.
4. Remove large empty spaces; no figure alone on a page with empty space around it.
5. Run the Word spell check; check page count (minimum 6, target 7– 9).
6. Export to PDF and look at every page.

Done when

- Every point of the documentation checklist (Appendix D) is true
- Zero mistakes found by the second reader
- 7–9 pages, PDF exported

Deliverables

Final document (PDF + source file)

React deliverable

Finalize figure numbering, alt descriptions or descriptive captions, and references. Confirm screenshots are current, synthetic and readable in the submitted PDF.

AI-agent prompt

Work on #47 (Documentation: formatting and final proofreading) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Finalize figure numbering, alt descriptions or descriptive captions, and references. Confirm screenshots are current, synthetic and readable in the submitted PDF. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#48 Rehearsal #1 (Milestone D: docs and slides ready)

Erti Hoxha, Isa Bilalli, Florent Latifi

Mon 30 Nov Needs #45 #47 1 h v4: Milestone D | v5.1 source p. 29

The first full run with a timer, while there is still time to fix the slides.

Task steps

1. Present the full deck with a timer; the target is 4:40 so we have 20 seconds of safety.
2. Each speaker practises their handover sentence.
3. Run the live demo once, then the backup video once.
4. Write down every slide that was too long or unclear and fix it the same evening.

Done when

• Total time under 5:00

• Document final and slides ready

Deliverables

Notes on what to fix

React deliverable

Rehearse the React demo from a clean start. Test the exact walkthrough, reset the demo dataset and keep the screenshot/video fallback accessible.

AI-agent prompt

Work on #48 (Rehearsal #1 (Milestone D: docs and slides ready)) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Rehearse the React demo from a clean start. Test the exact walkthrough, reset the demo dataset and keep the screenshot/video fallback accessible. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#49 FINAL CHECK: the project is finished

Erti Hoxha, Isa Bilalli, Florent Latifi

Tue 1 Dec Needs #48 1 h v4: Milestone Done | v5.1 source p. 30

The project is complete today. We check everything against the lists before submitting.

Task steps

1. Code: fresh clone → install → tests pass → python -m pft menu works.
2. Documentation: go through Appendix D point by point.
3. Slides: same design, no spelling mistakes, ≤ 30 words per slide.
4. Prepare the submission package: repository link (or ZIP of the frozen version), document PDF, slides PDF.
5. Everyone confirms in the chat: “I checked my part.”

Done when

- All checklists ticked
- Submission package ready
- Each member confirmed in the chat

Deliverables

Submission package

React deliverable

Clone main and follow the README to install Python and React, seed demo data, build both parts and test the full web flow. Verify package and screenshots match the tested commit.

AI-agent prompt

Work on #49 (FINAL CHECK: the project is finished) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Clone main and follow the README to install Python and React, seed demo data, build both parts and test the full web flow. Verify package and screenshots match the tested commit. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#50 Submit the project

Erti Hoxha

Wed 2 Dec Needs #49 0.5 h | v5.1 source p. 30

We submit 4 days before the official deadline (6 December, −3 points per late day).

Task steps

1. Upload the package in the way the professor asked.
2. Take a screenshot of the confirmation and post it in the chat.
3. Keep a copy of everything on two places (cloud + USB).

Done when

• Submission confirmed with a screenshot

Deliverables

Confirmation screenshot

React deliverable

Submit the project package with React source, lockfile, API setup, demo instructions and documentation. Exclude node_modules, credentials and personal financial data.

AI-agent prompt

Work on #50 (Submit the project) for Erti Hoxha. Read this task and its dependencies first. React deliverable: Submit the project package with React source, lockfile, API setup, demo instructions and documentation. Exclude node_modules, credentials and personal financial data. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

#51 After submission: defense preparation

Erti Hoxha, Isa Bilalli, Florent Latifi

Thu 3 Dec – Tue 8 Dec Needs #50 4 h v4: Guide v4 §15 | v5.1 source p. 30

21 of 40 points come from the individual defense: 3 questions per person, 7 points each, about any part of the project.

Task steps

1. Every day: study the 28 questions of the defense question bank (Guide v4, section 15) and answer each in 1–2 minutes while pointing to our code.

2. Thursday 3 December: rehearsal #2. Saturday 5 December: rehearsal #3.

3. Sunday 6 or Monday 7 December: mock defense. Each member draws 3 random numbers (1–28), answers without notes, and the others score 0–7. Any answer below 5 is studied again the same day.

4. Tuesday 8 December: presentation and defense.

Done when

• Each member scored at least 5/7 on every question in the mock defense

Deliverables

—

React deliverable

Practice explaining React state, API mapping, exact money, persistence and errors. Have each member demo one screen and trace one action into the Python code.

AI-agent prompt

Work on #51 (After submission: defense preparation) for Erti Hoxha, Isa Bilalli, Florent Latifi. Read this task and its dependencies first. React deliverable: Practice explaining React state, API mapping, exact money, persistence and errors. Have each member demo one screen and trace one action into the Python code. Preserve the Python domain and API contract. Use synthetic data. For a coordination or documentation task, draft the requested evidence without sending or publishing it. For code changes, inspect existing files, make the smallest relevant change and run the applicable checks. Use only main. Show the diff and test results for human review before any push.

Appendix A Requirements

FR-01 Create, rename and delete accounts (CASH, BANK, CARD, SAVINGS) with an opening balance Must Florent Latifi #7

FR-02 Log an income: amount, date, category, account, optional note Must Isa Bilalli #14

FR-03 Log an expense with the same fields Must Isa Bilalli #14

FR-04 List, edit and delete transactions by ID Must Florent Latifi #7 Add explicit edit support by ID in #07 and integrate it in #36.

FR-05 Balance of each account and total net worth Must Florent Latifi #11

FR-06 Filter by category, type, account, date range, amount range, note text; combinable Must Erti Hoxha #8

FR-07 Sort by date, amount or category, ascending or descending Should Erti Hoxha #15

FR-08 Monthly summary: income, expenses, net, savings rate, top 3 categories, comparison Must Florent Latifi #19

FR-09 Category report: EUR and % per category for any period Must Florent Latifi #26

FR-10 Trend report for the last N months Should Isa Bilalli #31

FR-11 Export transactions and reports to CSV Must Erti Hoxha #16

FR-12 Export to JSON and Markdown (new format = new class) Should Erti Hoxha #24

FR-13 Save automatically after every change; load on start Must Florent Latifi #18

FR-14 Transfers between accounts (not income or expense) Should Isa Bilalli #23

FR-15 Monthly budget per category Should Isa Bilalli #27

FR-16 Warnings at 80% and 100% of a budget (Observer) Should Isa Bilalli #27

FR-17 Budget report: planned vs. actual Should Isa Bilalli #31

FR-18 Recurring transactions created automatically Should Isa Bilalli #25

FR-19 Import a bank statement CSV with error list Could As decided #28

FR-20 Undo and redo Could As decided #28

FR-21 Charts as PNG (matplotlib) Could As decided #28

FR-22 CLI sub-commands and interactive menu Must Florent Latifi #21 Keep the CLI requirement until an API replacement is approved.

FR-23 Clear message for every invalid input; never a traceback Must Isa Bilalli #10

Appendix B Repository and data

pft/domain/ holds entities, Money, transactions, validators and exceptions. pft/services/ holds Ledger, filters, reports and budget monitoring. pft/infrastructure/ holds repository and export adapters. pft/api/ contains FastAPI routes and DTO mapping. frontend/ contains the Vite app, routes, shared components, API client and tests.

Keep tests/, docs/, data/, README.md, pyproject.toml and .github/workflows/ci.yml. Keep local data, virtual environments and node_modules out of Git.

SQLite database and CSV baseline

Task #05 configures the local SQLite file and initialization. Task #18 implements domain persistence through SqliteRepository. Keep any CSV capabilities required by the course.

Accounts carry id, name, type, opening balance, currency and created date. Transactions carry id, account id, kind, exact amount, currency, date, category, note and optional transfer ID/direction. Budgets use category, month, limit and currency. Recurring rules carry account, kind, amount, category, note, frequency, dates and run tracking.

Use ISO dates. The amount is positive and kind/direction determines sign. Both sides of a transfer share an ID and commit together. SQLite keeps exact values as integer cents or decimal text. Use a connection scoped to the operation, transactions for writes and synchronization if Ledger state is shared.

Reference documents

V5.1 Personal Finance Tracker – Step-by-Step Task Plan, 9 November 2026. The original sketches, FR and NFR codes, and course requirements are the baseline for this revision.

Kickoff discussion, 11 November 2026. The React GUI scope, FastAPI boundary and SQLite database inform the web implementation.

Appendix C Submission documentation

1. Introduction: problem, goals, scope and user journey (#09).

2. Solution / Approach / Architecture: requirements, Python layers, API and React request flow (#20, #33, #41).

3. Implementation: domain, storage, API, React components, screenshots, tests and coverage (#29, #34, #42).

4. Conclusion: results, limits and future work (#43).

5. References: sources used in the work (#43).

The v5.1 baseline specifies Times New Roman 10 pt, at least six pages, six numbered figures, an IEEE reference list and proofreading. Confirm the professor’s actual template, language and reference requirements before submission.

Target 7–9 pages. Caption and cite each figure in the text. Cite every listed source and use consistent terms, tense, dates and number formats. Use spell check and two readers. Keep defense slides to 30 words each where practical, use code at least 18 pt and rehearse the demo.